

Unidade I

1 FUNDAMENTOS DO DESENVOLVIMENTO WEB

1.1 Introdução ao HTML5 e elementos semânticos

O HTML5 é a espinha dorsal da web moderna. Ele não é apenas uma linguagem de marcação para organizar conteúdo, mas sim a base estrutural que define como navegadores, mecanismos de busca, leitores de tela e outras tecnologias interpretam, exibem e interagem com documentos digitais.

HTML 3.2 e HTML 4.01, já permitiam organizar textos, criar links e inserir imagens, mas eram limitadas quando se tratava de semântica, acessibilidade e integração com tecnologias modernas. Em muitos casos, desenvolvedores precisavam recorrer a “gambiarras” ou a recursos externos, como Flash e plugins proprietários, para implementar áudio, vídeo, animações e formulários avançados.

O HTML5 surge, então, como resposta a essas limitações. Seu objetivo é claro: fornecer uma linguagem capaz de estruturar o conteúdo de forma semântica, padronizada e otimizada para o ecossistema digital atual, que inclui computadores, tablets, smartphones, televisores inteligentes e até dispositivos de realidade aumentada.

HTML5 é a semântica. Antes dele, era comum o uso excessivo de elementos genéricos, como `<div>` e `<p>`, que não tinham qualquer significado intrínseco, o que tornava o código menos legível.

O HTML5 introduziu uma série de novas tags semânticas que ajudam a identificar o papel de cada parte do conteúdo.

`<header>`: representa o cabeçalho de uma página ou seção, geralmente contendo títulos, logotipos e menus principais.

`<nav>`: define áreas de navegação, com links importantes que ajudam o usuário a se deslocar pelo site.

`<main>`: delimita o conteúdo principal de uma página, diferenciando-o de barras laterais ou rodapés.

`<article>`: representa um conteúdo independente, como uma notícia, publicação em fórum ou post de blog.

`<section>`: agrupa conteúdos relacionados dentro de uma página, organizando-os logicamente.

<aside>: indica conteúdos secundários ou complementares, como barras laterais, widgets ou links extras.

<footer>: representa o rodapé da página, usualmente com créditos, informações legais e links adicionais.

Essa estrutura mais clara melhora tanto a leitura do código por humanos quanto a interpretação automática por máquinas. Leitores de tela, usados por pessoas com deficiência visual, conseguem navegar de forma mais precisa; mecanismos de busca, como o Google, interpretam melhor a relevância do conteúdo, impactando diretamente o Search Engine Optimization (SEO).

<audio>: possibilita a reprodução de sons, podcasts e músicas de forma simples.

<video>: permite incorporar vídeos com controles nativos, como play, pause e volume.

<canvas>: fornece uma área para renderização de gráficos dinâmicos e jogos interativos usando JavaScript.

<svg>: viabiliza gráficos vetoriais escaláveis diretamente no navegador.

• Tipos de input novos: e-mail, url, number, date, time, range, entre outros. • Atributos inteligentes: required, placeholder, autofocus, pattern. • Validação nativa: navegadores agora conseguem validar automaticamente se um campo de e-mail contém um @ ou se um campo numérico está dentro de um intervalo definido.

2 ESTILOS E DESIGN COM CSS3

2.1 Introdução ao CSS3 e suas propriedades visuais

O CSS3 é a linguagem utilizada para definir a aparência visual e a experiência estética das páginas web. Enquanto o HTML5 é responsável pela estrutura e organização do conteúdo, o CSS3 assume o papel de estilizar os elementos, trazendo cor, harmonia e identidade visual ao projeto.

2.2 O que o CSS3 trouxe de novo em relação às versões anteriores

Antes do CSS3, estilizar páginas era um processo limitado. Desenvolvedores precisavam recorrer a imagens para criar efeitos como gradientes, sombras ou cantos arredondados.

• Bordas arredondadas (border-radius): antes feitas com imagens, agora podem ser criadas apenas com código. • Sombras (box-shadow e text-shadow): permitem dar profundidade e destaque a elementos e textos. • Gradientes (linear-gradient, radial-gradient): substituem imagens pesadas e oferecem infinitas combinações de cores. • Transformações

e animações (transform, transition, animation): possibilitam criar efeitos dinâmicos e fluidos. • Media queries: recurso essencial para a responsividade, permitindo que o layout se adapte a diferentes tamanhos de tela. • Flexbox e grid layout: novas formas de organizar elementos, tornando a construção de layouts mais intuitiva e eficiente.

2.3 Importância do CSS3 para o design moderno

Graças às media queries e novas unidades de medida, como vh, vw, rem, fr, é possível construir sites que se adaptam automaticamente a diferentes resoluções. A responsividade deixou de ser um diferencial e se tornou um requisito básico no desenvolvimento web moderno. O CSS3 foi essencial para essa mudança, abrindo caminho para frameworks como Bootstrap e Tailwind CSS, que exploram ao máximo suas capacidades.

Os seletores CSS são padrões usados para indicar quais elementos de uma página HTML devem receber determinados estilos. Em outras palavras, são como “endereços” que apontam para os elementos desejados dentro da estrutura do documento. A partir dessa seleção, o CSS aplica propriedades de cor, fonte, espaçamento, alinhamento, entre outras.

exemplo: `p { color: blue }`

O Box Model é um conceito fundamental no CSS, que define como os elementos são renderizados na página. Ele consiste em: • Conteúdo (content): o conteúdo real do elemento (texto, imagem etc.). • Preenchimento (padding): espaço entre o conteúdo e a borda. • Borda (border): a linha ao redor do preenchimento. • Margem (margin): espaço externo ao elemento.

2.4 Aplicação de cores, gradientes e animações

- Nomes de cores (keywords): o CSS tem uma lista de cores pré-definidas, como red, blue, green, black, white
- Hexadecimal (#RRGGBB): forma tradicional de representar cores, combinando valores de vermelho, verde e azul
- RGB (Red, Green, Blue): define cores com base na intensidade de cada canal
- RGBA (Red, Green, Blue, Alpha): similar ao RGB, mas com um quarto parâmetro que controla a transparência
- HSL (Hue, Saturation, Lightness): representa cores de forma mais intuitiva, com matiz (hue), saturação e luminosidade.
- HSLA (Hue, Saturation, Lightness, Alpha): versão com transparência do modelo HSL.

Gradiente linear

O gradiente é aplicado em linha reta (vertical, horizontal ou diagonal).

Gradiente cônico (conic gradient)

Introduzido recentemente, cria gradientes em formato circular contínuo. É Ideal para gráficos circulares ou efeitos criativos.

Botões chamativos

```
.card{  
    background: radial-gradient(circle at top left, #fff, #f0f0f0)  
}
```

CSS3 permite criar animações e transformações:

```
1  /* Transformações 2D */  
2  .transformar {  
3      transform: translate(50px, 100px); /* Move o elemento */  
4      transform: rotate(30deg);          /* Rotaciona o elemento */  
5      transform: scale(2, 3);             /* Escala o elemento */  
6      transform: skew(20deg, 10deg);      /* Inclina o elemento */  
7  }  
8  
9  /* Transições */  
10 .transicionar {  
11     transition: all 0.3s ease-in-out;  
12 }  
13  
14 .transicionar:hover {  
15     transform: scale(1.1);  
16     background-color: #f0f0f0;  
17 }  
18  
19 /* Animações */  
20 @keyframes exemplo {  
21     0% { opacity: 0; }  
22     100% { opacity: 1; }  
23 }  
24  
25 .animar {  
26     animation-name: exemplo;  
27     animation-duration: 2s;  
28     animation-iteration-count: infinite;  
29 }
```

Enquanto a acessibilidade concentra-se na inclusão de pessoas com deficiência e na adaptação para diferentes dispositivos, o SEO preocupa-se com a visibilidade nos mecanismos de busca.

A acessibilidade web significa que qualquer pessoa, independentemente de suas habilidades, pode perceber, entender, navegar e interagir com uma página ou aplicação.

W3C (World Wide Web Consortium) desenvolveu as WCAG (Web Content Accessibility Guidelines, POUR:

- Perceptível (perceivable): as informações devem estar disponíveis de forma que possam ser percebidas por todos os sentidos, seja visão, audição ou tato.
- Operável (operable): todos os elementos devem ser utilizáveis por diferentes formas de navegação, incluindo teclado, leitores de tela e comandos de voz.
- Compreensível (understandable): o

conteúdo deve ser claro, os formulários fáceis de preencher e a navegação previsível • • Robusto (robust): o site deve ser compatível com diferentes tecnologias assistivas, navegadores e dispositivos.

Para aplicar acessibilidade de forma prática, algumas ações são fundamentais no desenvolvimento web moderno: • HTML semântico: utilizar corretamente tags como , , , e garante que leitores de tela compreendam a hierarquia e o significado do conteúdo. • Texto alternativo em imagens (alt): uma imagem sem descrição alternativa é invisível para pessoas com deficiência visual. O atributo alt deve descrever o conteúdo ou a função da imagem.

- Navegação por teclado: todos os elementos interativos devem ser acessíveis apenas com Tab, Enter e setas, o que é essencial para pessoas que não usam mouse.

```
<button tabindex="0">Enviar</button>
```

- Uso de ARIA: fornece atributos extras para descrever componentes dinâmicos, como menus, carrosséis e modais, garantindo que leitores de tela informem o comportamento ao usuário.

```
<nav role="navigation" aria-label="Menu principal">
```

O SEO é o conjunto de técnicas voltadas para otimizar páginas web a fim de alcançar melhores posições nos mecanismos de busca, como Google e Bing. Quanto melhor o SEO, maior a probabilidade de o site ser encontrado pelo público-alvo. Assim como a acessibilidade, o SEO valoriza estrutura, clareza e performance.

Por sua vez, as meta tags são instruções colocadas no da página HTML. Elas não aparecem para o usuário, mas são interpretadas pelos buscadores. Exemplos comuns incluem: • Metatítulo e descrição: definem como a página aparecerá nos resultados de busca. • Meta keywords: embora pouco usadas hoje, ainda são relevantes em alguns buscadores menores. • Meta viewport: essencial para responsividade.

- Meta robots: define como os mecanismos de busca devem indexar a página. • Meta Open Graph e Twitter Cards: usadas para otimizar a forma como a página aparece em redes sociais.

2.5 Acessibilidade e SEO: uma relação direta

Muitas práticas de acessibilidade impactam diretamente o SEO, como: • Um site bem estruturado semanticamente é melhor interpretado pelos buscadores. • O uso de texto alternativo em imagens melhora a acessibilidade e também gera resultados em Google Imagens. • A responsividade beneficia tanto pessoas com deficiência quanto o ranqueamento no Google. • A velocidade de carregamento, importante para SEO, também auxilia usuários com conexões lentas.

Unidade II

3 INTRODUÇÃO AO DESIGN RESPONSIVO

3.1 O que é design responsivo

O objetivo é proporcionar uma experiência de navegação ideal em qualquer dispositivo, seja ele um desktop tradicional com tela widescreen, um tablet em modo retrato ou paisagem ou até mesmo um smartphone com tela reduzida.

3.2 Experiência do Usuário (UX)

. Desde 2015, a gigante da tecnologia anunciou o mobile first indexing, isto é, a versão móvel de um site passa a ser a principal referência para ranqueamento: • sites não responsivos perdem posições nos resultados de busca; • páginas rápidas e responsivas ganham prioridade; • a usabilidade é um fator direto de SEO técnico.

3.3 Acessibilidade e estatísticas

Pesquisas recentes confirmam que mais de 60% do tráfego mundial da internet vem de dispositivos móveis. Em alguns setores, como e-commerce e redes sociais, essa porcentagem ultrapassa 80% (Araujo, 2024).

3.4 Princípios do design responsivo

- %: define o tamanho com base na porcentagem do elemento pai;
- em: relacionado ao tamanho da fonte do elemento pai;
- rem: relacionado ao tamanho da fonte do elemento raiz (html);
- vw / vh: define dimensões com base na largura (viewport width) e altura (viewport height) da tela.

3.5 Imagens flexíveis

- max-width: 100%: impede que a imagem ultrapasse os limites do container;
- height: auto: mantém a proporção original da imagem, evitando distorções.

3.6 Mobile-first

Essa abordagem consiste em desenvolver o design pensando primeiro na experiência dos dispositivos móveis. Depois, aplicamos media queries para expandir o layout para telas maiores, como tablets e desktops.

- A maioria dos acessos à internet vem de dispositivos móveis (mais de 60% do tráfego mundial).
- Sites desenvolvidos pensando no mobile têm melhor desempenho em SEO, pois o Google prioriza conteúdos otimizados para telas pequenas.
- É mais fácil construir do pequeno para o grande do que adaptar um layout complexo para telas menores.

4 MEDIA QUERIES E BREAKPOINTS

Elas são uma funcionalidade crucial do CSS3 que permite aplicar estilos condicionalmente com base nas características do dispositivo onde o conteúdo está sendo exibido.

sintaxe:

```
@media [tipo de mídia] and (condicao) {  
    /* css estilo */  
}
```

4.2 Uso de media queries para diferentes dispositivos

O tipo all é o mais abrangente e, como o próprio nome sugere, aplica os estilos a qualquer tipo de mídia

Quando usar: • para criar layouts responsivos universais; • para manter um padrão visual único entre diferentes dispositivos; • screen: destinado principalmente a telas coloridas.

O tipo screen é o mais utilizado no desenvolvimento web moderno. Ele é específico para dispositivos com telas, como monitores, notebooks, tablets e smartphones. Esse tipo de mídia é ideal quando você quer aplicar estilos apenas para exibição digital, sem afetar a visualização em modo de impressão ou outros formatos.

Quando usar: • para sites responsivos voltados para telas; • para aplicar estilos dinâmicos e layouts adaptáveis; • quando o objetivo é personalizar a experiência visual para usuários digitais; • print: para documentos impressos ou visualização em modo de impressão.

O tipo print é usado quando queremos aplicar estilos específicos para impressão. Ele é muito útil para relatórios, certificados, documentos ou páginas que podem ser impressas

Quando usar: • para criar versões otimizadas para impressão; • para remover menus, anúncios, animações e conteúdos não essenciais; • para economizar tinta e melhorar a legibilidade; • speech: para sintetizadores de voz.

O tipo speech é projetado para tecnologias assistivas que transformam o conteúdo em áudio, como leitores de tela para pessoas com deficiência visual. Esse tipo de mídia permite criar estilos que melhoram a acessibilidade, controlando como o conteúdo será narrado. A figura a seguir ilustra um exemplo de uso:

Quando usar: • para criar versões otimizadas para impressão; • para remover menus, anúncios, animações e conteúdos não essenciais; • para economizar tinta e melhorar a legibilidade; • speech: para sintetizadores de voz;

Width – largura exata da viewport

Height – altura exata da viewport

min-width – largura mínima da viewport  (mais usado)

max-width – largura máxima da viewport  (muito usado)

• Portrait: quando a altura é maior que a largura (modo vertical). • Landscape: quando a largura é maior que a altura (modo horizontal)

Hover – capacidade de passar o mouse.

Pointer – precisão do dispositivo apontador

Resolution – resolução da tela

Aspect-ratio – proporção da tela

4.3 Breakpoints baseados em conteúdo

A abordagem mais eficaz para definir breakpoints é baseá-los no conteúdo, não em dispositivos específicos. Isso significa ajustar o layout quando o conteúdo começa a parecer desorganizado, independentemente do dispositivo.

4.4 Princípios de design responsivo

A consistência visual é um dos pilares do UX Design (User Experience Design) e também um elemento-chave no design responsivo. Quando um usuário acessa um site, seja em um computador de mesa, um tablet ou um smartphone, ele espera encontrar uma identidade visual uniforme

A boa prática diz: o tamanho mínimo recomendado para botões é 44px x 44px (padrão da Apple e do Google). Em smartphones, o hover não funciona. Por isso, o feedback visual deve acontecer no clique.

```
1  /* Para dispositivos com mouse */
2  @media (hover: hover) {
3      .botao:hover {
4          background-color: #ff6347;
5      }
6  }
7
8  /* Para dispositivos com toque */
9  @media (hover: none) {
10     .botao:active {
11         background-color: #ff4500;
12     }
13 }
```

- em telas touch, os elementos devem ter mais espaço entre si;
- evite menus e botões muito próximos, pois isso gera toques acidentais;
- em telas desktop, a densidade pode ser maior, aproveitando melhor o espaço.

- UX: garante que a jornada do usuário seja fluida, sem obstáculos desnecessários, priorizando a clareza, a acessibilidade e a satisfação.
- UI: cuida da parte visual, garantindo que cada elemento seja agradável, bem posicionado e consistente com a identidade da marca.
- CSS responsivo: é a ponte que une UX e UI, tornando possível a adaptação visual em qualquer dispositivo.

4.6 Padrões comuns de navegação responsiva

O menu hamburger é, sem dúvida, um dos padrões mais populares e difundidos no design responsivo, especialmente em interfaces móveis. Seu nome vem da semelhança visual do ícone com um “hambúrguer”: três linhas horizontais paralelas que lembram o pão, a carne e novamente o pão. Esse ícone, geralmente localizado no canto superior esquerdo ou direito da tela, se tornou praticamente um símbolo universal de navegação compacta.

4.7 Padrão Priority+

O padrão Priority+ é uma abordagem de navegação responsiva que busca equilibrar clareza, usabilidade e economia de espaço. A ideia central é priorizar os itens mais importantes na barra principal de navegação, deixando visíveis apenas os links que representam maior valor para o usuário ou para os objetivos do site.

Os itens considerados menos relevantes ou acessados com menor frequência são agrupados em um menu suspenso, geralmente representado por três pontos (...), pela palavra “Mais” ou por um ícone similar.

4.8 Dropdown (menus suspensos)

O padrão Dropdown, também conhecido como menu suspenso, é um dos elementos de navegação mais tradicionais e difundidos no design de interfaces digitais. Ele funciona como um menu principal que, ao ser clicado, tocado ou ao receber o cursor do mouse (hover), exibe subcategorias ou opções adicionais relacionadas.

4.9 Footer navigation (navegação no rodapé)

Trata-se de um padrão amplamente usado em sites responsivos e aplicativos móveis. Ele consiste em organizar links importantes e elementos de navegação na parte inferior da página, seja em forma de barra fixa em dispositivos móveis ou como um bloco de links mais extensos em sites desktop.

4.10 Outros padrões de navegação responsiva

Navegação por abas (tabbed navigation)

Trata-se de um padrão extremamente popular, principalmente em aplicativos móveis. Ela consiste em exibir as opções principais na parte inferior da tela, geralmente acompanhadas de ícones e rótulos curtos

Navegação lateral (off-canvas menu)

É ativado geralmente por meio de um botão (frequentemente o ícone de hamburger). Quando clicado ou tocado, o menu desliza lateralmente para dentro da tela, revelando opções adicionais de navegação.

Sticky navigation (navegação fixa)

Mantém a barra de navegação sempre visível no topo ou rodapé da tela, mesmo quando o usuário rola a página para baixo. Esse padrão é bastante usado em portais de conteúdo, dashboards e e-commerces, onde o acesso rápido a diferentes seções é essencial.

Unidade III

5 LAYOUT FLEXÍVEL COM CSS, FLEXBOX E GRID

5.1 Introdução ao sistema de layout flexbox

O flexbox trouxe a capacidade de organizar elementos em uma dimensão (linha ou coluna), alinhando, distribuindo e ordenando itens com extrema facilidade. Já o CSS grid foi além, oferecendo um sistema bidimensional, no qual é possível controlar colunas e linhas de maneira integrada e eficiente.

O flexbox (flexible box layout) é um módulo de layout do CSS3 que permite distribuir espaço e alinhar itens de maneira eficiente, mesmo quando os tamanhos dos elementos são desconhecidos ou dinâmicos. Diferente dos métodos de layout tradicionais, como float e inline-block, o flexbox foi desenvolvido para simplificar a criação de layouts complexos, tornando o posicionamento e o dimensionamento de elementos mais previsível e flexível.

Um dos principais conceitos do flexbox é o container flexível. Para transformar um elemento em um container flex, utiliza-se a propriedade:

```
.container {  
    display: flex  
}
```

Flexbox é ideal para layouts unidimensionais; CSS grid é mais eficiente para grades e estruturas complexas.

5.2 Alinhamento, distribuição e ordenação dos elementos

- justify-content: controla o alinhamento horizontal dos itens no container;
- align-items: controla o alinhamento vertical dos itens em relação ao eixo transversal;
- align-content: controla o espaçamento entre linhas quando há múltiplas linhas.

A flexibilidade de cada item é controlada pela propriedade flex, que combina três valores: crescimento (flex-grow), encolhimento (flex-shrink) e base (flex-basis).

flex: 1 => ocupa espaço igual aos outros itens flexíveis

flex-wrap permite que os itens “quebrem” para uma nova linha quando não houver espaço suficiente

5.3 Construção de layouts unidimensionais adaptáveis

O flexbox permite:

- Criar estruturas unidimensionais (em linhas ou colunas) com total controle sobre alinhamento e espaçamento.
- Centralizar elementos de maneira simples, algo que antes exigia linhas de código complexas.
- Construir componentes reutilizáveis, como cards, galerias, barras de navegação e rodapés flexíveis.
- Ajustar automaticamente o layout em diferentes resoluções, favorecendo a responsividade.

5.4 Propriedades do container Flex

O container Flex é o elemento que envolve os itens flexíveis e define como eles serão distribuídos, alinhados e dimensionados dentro do espaço disponível.

- Flex: o container se torna um bloco flexível, ocupando toda a largura disponível e criando um contexto para os itens filhos.
- Inline-flex: o container se comporta como um elemento inline, ocupando apenas o espaço necessário, mas ainda permitindo que os filhos sejam tratados como flexíveis.

Essa propriedade é essencial, pois sem ela as demais propriedades do flexbox não terão efeito. Por sua vez, o flex-direction define a direção principal do layout dentro do container flex:

- row (padrão): os itens são distribuídos da esquerda para a direita;
- row-reverse: os itens são distribuídos da direita para a esquerda;
- column: os itens são empilhados de cima para baixo;
- column-reverse: os itens são empilhados de baixo para cima.

- Nowrap (padrão): todos os itens ficam em uma única linha, mesmo que extrapolem o container.
- Wrap: os itens quebram para a próxima linha quando necessário.
- Wrap-reverse: os itens quebram para a próxima linha, mas a nova linha aparece acima da anterior.

O justify-content define o alinhamento dos itens no eixo principal (horizontal para row). Os valores mais comuns são:

- flex-start: itens alinhados ao início do container;
- flex-end: itens alinhados ao final do container;
- center: itens centralizados;
- space-between: espaço igual entre os itens, sem espaço nas extremidades;
- space-around: espaço igual entre os itens, com metade do espaço nas extremidades;
- space-evenly: espaço igual entre itens e nas extremidades.

A respeito do align-items, considera-se que o alinhamento ocorre dos itens no eixo transversal (vertical para row). Os valores principais incluem:

- stretch (padrão): itens se esticam para preencher a altura do container;
- flex-start: alinhamento no início do eixo transversal;
- flex-end: alinhamento no final do eixo transversal;
- center: centraliza os itens verticalmente;
- baseline: alinha os itens pela linha de base do texto.

align-content controla o alinhamento de múltiplas linhas de itens quando o container tem flex-wrap: wrap. Os valores possíveis são:

- flex-start: linhas agrupadas no início do container;
- flex-end: linhas agrupadas no final do container;
- center: linhas centralizadas;

space-between: espaço igual entre linhas, sem espaço nas extremidades; • space-around: espaço igual entre linhas, com metade do espaço nas extremidades; • stretch (padrão): linhas se esticam para preencher o container.

Gap

Define o espaçamento entre itens do container, substituindo a necessidade de usar margens individuais:

- flex-grow: 0 – o item não cresce além do seu tamanho inicial; • flex-grow: 1 – o item cresce igualmente aos outros itens com valor 1; • flex-grow: 2 – o item cresce duas vezes mais que um item com valor 1.

- flex-shrink: 1 (padrão) – o item pode encolher proporcionalmente aos outros itens; • flex-shrink: 0 – o item não encolhe, mantendo seu tamanho mínimo.

Flex-basis define o tamanho inicial do item antes de qualquer crescimento (flex-grow) ou encolhimento (flex-shrink).

6 LAYOUTS FLUIDOS E PROPORCIONAIS

O layout fluido se apoia em unidades relativas e proporcionais, permitindo que os elementos da página se adaptem naturalmente a qualquer resolução de tela.

6.2 Como funciona um layout fluido: benefícios e desafios

- Unidades percentuais (%): definem tamanhos de acordo com o espaço disponível. Exemplo: uma imagem com width: 50% sempre ocupará metade do container, independentemente da largura total da tela.
- Unidades relativas à fonte (em, rem): permitem que tamanhos de texto e espaçamentos cresçam ou diminuam proporcionalmente, mantendo a harmonia visual.
- Unidades relativas ao viewport (vh, vw): ajustam dimensões de acordo com a largura ou altura da janela de visualização.
- Funções modernas como clamp(): permitem definir valores mínimos, ideais e máximos, garantindo controle total sobre a fluidez.

6.3 Futuro dos layouts fluidos

% (percentual)

Representa a largura ou altura em relação ao elemento pai. Ideal para containers e elementos que precisam se adaptar ao espaço disponível.

vw / vh (viewport width / viewport height)

Percentual relativo à largura ou altura da viewport, ou seja, da tela do dispositivo.

em / rem

Em: é relativo ao tamanho da fonte do elemento pai. Rem: é relativo ao tamanho da fonte do elemento raiz (html).

clamp(), min() e max()

Essas funções permitem limitar valores entre mínimo e máximo, mantendo comportamento fluido.

Explicação prática

O elemento ajusta sua largura e altura de acordo com o tamanho da tela, mantendo limites mínimos e máximos predefinidos, o que garante consistência visual, mesmo em resoluções extremas.

O Aspect Ratio Box cria uma caixa proporcional, útil para imagens e vídeos:

Enquanto o flexbox se destaca por sua capacidade de alinhar elementos em uma única dimensão (ou linha ou coluna), o CSS grid se apresenta como a solução definitiva para organizar interfaces de forma mais sofisticada, possibilitando combinações de linhas e colunas que criam estruturas flexíveis, escaláveis e precisas.

O grid é baseado em dois eixos principais: o horizontal (linhas), e o vertical (colunas). Com ele, é possível criar grades personalizadas, definindo: • quantas colunas e linhas um layout terá; • o tamanho de cada uma dessas áreas; • o espaçamento entre elas (gap); • como os elementos se alinham dentro da grade; • áreas inteiras que podem ser nomeadas e reutilizadas.

```
1  .container {
2    display: grid;
3    grid-template-columns: 1fr 2fr 1fr;
4    grid-template-rows: auto 300px auto;
5    gap: 20px;
6  }
7
```

Nesse caso, temos um layout com três colunas (a primeira e a terceira ocupando o mesmo espaço, enquanto a segunda é duas vezes maior) e três linhas (a primeira e a última ajustadas automaticamente e a linha central fixa em 300px).

• fr (fractional unit): unidade exclusiva do grid que distribui o espaço disponível proporcionalmente; • auto: ajusta automaticamente de acordo com o conteúdo; • minmax(): define um valor mínimo e máximo para cada linha ou coluna;

- `repeat()`: simplifica a repetição de colunas ou linhas;
- `fit-content()`: ajusta a largura ou altura ao tamanho do conteúdo, respeitando limites

- Flexbox: ideal para alinhamento e distribuição em uma dimensão; é excelente para componentes, como menus, cards ou formulários.
- CSS grid: ideal para organização de layouts completos em duas dimensões; é perfeito para estruturas maiores, como a página inteira.

Quando usamos a configuração `contain: layout style paint`, estamos indicando explicitamente que:

- O elemento gerencia seu próprio layout, não influenciando a posição de elementos externos.
- O elemento isola seus estilos, reduzindo a necessidade de reprocessar regras CSS em outros blocos da página.
- O elemento é responsável pela sua própria pintura, evitando que alterações visuais internas forcem um redesenho de áreas maiores do documento.

A propriedade `will-change` é um recurso avançado do CSS que permite ao desenvolvedor informar antecipadamente ao navegador quais propriedades de um elemento provavelmente sofrerão mudanças. Essa indicação dá ao navegador a oportunidade de preparar otimizações internas antes da alteração acontecer, o que resulta em mais fluidez e em melhor desempenho em transições e animações. Em outras palavras, o `will-change` funciona como um aviso prévio ao mecanismo de renderização. Ele sinaliza ao navegador que determinado elemento sofrerá mudanças em suas propriedades visuais (como `transform`, `opacity`, `top`, `left` etc.), incentivando-o a criar camadas de composição independentes para esses elementos. Assim, quando a modificação realmente ocorrer, o navegador já estará preparado, reduzindo a necessidade de cálculos pesados de reflow (reorganização do layout) e repaint (repintura da tela).

Sem o `will-change`, o navegador trabalha de forma reativa: ele só otimiza um elemento quando detecta a alteração. Com o `will-change`, o navegador passa a ser proativo, otimizando o elemento antes da mudança acontecer

```
.card{  
  will-change: transform, opacity;  
}
```

Aqui, o navegador já sabe que o card sofrerá transformações (`transform`) e alterações de opacidade (`opacity`). Dessa forma, ele cria uma camada de renderização separada para o card, garantindo que animações futuras sejam executadas de forma mais fluida.

A propriedade `content-visibility` é um recurso moderno do CSS que representa um dos maiores avanços recentes em termos de performance de renderização. Seu funcionamento é simples, mas extremamente poderoso: ela permite que elementos sejam processados e renderizados pelo navegador apenas quando se tornam visíveis na tela do usuário. Em termos práticos, isso significa que, em vez de carregar e desenhar todos os elementos de uma página longa ou complexa logo no início, o navegador pode ignorar temporariamente os blocos de conteúdo que estão fora da área visível (viewport). Assim, o carregamento inicial fica mais rápido, já que a máquina do usuário não precisa gastar recursos desnecessários processando elementos que ainda não serão visualizados.

Unidade IV

7 FRAMEWORKS PARA DESENVOLVIMENTO

Frameworks front-end surgiram para:

- acelerar o desenvolvimento, oferecendo estruturas pré-prontas;
- garantir compatibilidade entre navegadores e dispositivos;
- promover padrões de design, deixando o produto final mais coerente;
- melhorar a manutenção, já que equipes inteiras trabalham com convenções comuns

7.2 Criação de layouts responsivos com grid system

O grid system é o coração do Bootstrap. Ele utiliza uma divisão em 12 colunas, que podem ser combinadas em diferentes larguras.

- container (`.container` ou `.container-fluid`): define a área central;
- linha (`.row`): agrupa colunas;
- colunas (`.col-*`): definem a largura responsiva.

O Bootstrap oferece uma grande variedade de componentes prontos, como:

- Navbar: navegação responsiva que colapsa em menu hamburger em telas menores.
- Cards: blocos versáteis para apresentar conteúdos e imagens.
- Modal: janelas de diálogo elegantes para exibir informações adicionais.
- Carousel: carrosséis de imagens ou conteúdo dinâmico.

8 JAVASCRIPT E INTERATIVIDADE

JavaScript é o responsável por interação, dinamismo e comportamento.

8.1 JavaScript para dinamismo e interatividade

- ES6 (2015) trouxe `let`, `const`, arrow functions, classes e módulos.
- Hoje navegadores modernos oferecem APIs poderosas, como Fetch, Web Animations, Web Components, entre outras

8.2 Criação de funcionalidades dinâmicas e eventos DOM

O DOM (Document Object Model) é a representação em árvore da página HTML. Manipular o DOM significa alterar o conteúdo.

Essa API é nativa, com controle de pausa, reversão e eventos de término. A Fetch API é a maneira moderna de fazer requisições HTTP.

GET

```
1  async function carregarDados() {
2    const resp = await fetch('/api/produtos');
3    const dados = await resp.json();
4    console.log(dados);
5  }
```

Figura 187 – Estrutura de pastas de projeto pronta para deploy

Post

```
1  async function enviarFormulario(data) {
2    await fetch('/api/form', {
3      method: 'POST',
4      headers: { 'Content-Type': 'application/json' },
5      body: JSON.stringify(data)
6    });
7  }
```

Figura 188 – Arquivo package.json com dependências configuradas

São também técnicas de performance para projetos complexos: • debounce/throttle em eventos de scroll e resize; • lazy loading de imagens para economizar banda; • uso de requestAnimationFrame em loops de animação.

```
1  function debounce(fn, delay){
2    let timeout;
3    return (...args) => {
4      clearTimeout(timeout);
5      timeout = setTimeout(() => fn(...args), delay);
6    };
7  }
8  window.addEventListener('resize', debounce(() => {
9    console.log('Redimensionou!');
10 }, 300));
```

Por sua vez, os Web Components permitem criar elementos HTML personalizados com encapsulamento de estilo, o que viabiliza a criação de componentes reutilizáveis sem dependência de frameworks.

```
1  class MeuCard extends HTMLElement {
2    connectedCallback() {
3      this.innerHTML = `<div class="card"><slot></slot></div>`;
4    }
5  }
6  customElements.define('meu-card', MeuCard);
7  |
```

```
1  const email = document.querySelector('#email');
2  email.addEventListener('input', () => {
3    if(!email.value.includes('@')){
4      email.classList.add('is-invalid');
5    } else {
6      email.classList.remove('is-invalid');
7    }
8  });
```

Tailwind CSS e Materialize / Tailwind CSS – Filosofia Utility-First

- você aplica classes utilitárias diretamente nos elementos;
- não é necessário escrever CSS adicional para boa parte das customizações;
- o design é altamente personalizável e escalável.

Padrões visuais e responsividade Tailwind utiliza breakpoints simples:

- sm ≥ 640px
- md ≥ 768px
- lg ≥ 1024px
- xl ≥ 1280px
- 2xl ≥ 1536px

- Code splitting: divisão de pacotes JavaScript, carregando somente o necessário em cada rota.
- Critical CSS: carregar apenas estilos essenciais no primeiro carregamento.
- Lazy loading: carregar imagens, vídeos e componentes somente quando visíveis.
- Minificação e compressão: reduzir tamanho de CSS/JS com ferramentas como Webpack e Vite.
- CDN e Caching: distribuir recursos em servidores globais, reduzindo latência.

Tele Aulas e Questionários

Pergunta 1

0 em 0 pontos



Qual das tags abaixo é um elemento semântico do HTML5?

Resposta Seleccionada:

✓ c.
<header>

Pergunta 2

0 em 0 pontos



O front-end é responsável pelo que em um site?

Resposta Seleccionada:

✓ c.
Pela estética e pelo visual do site.

Pergunta 3

0 em 0 pontos



O que é o VS Code?

Resposta Seleccionada:

✓ d.
Um editor de código.

Pergunta 4

0 em 0 pontos



Com base nos conteúdos estudados nesta unidade, qual é o principal objetivo do desenvolvimento front-end?

Resposta
Selecionada:



c.

Desenvolver a parte visual, interativa e acessível das páginas web.

Pergunta 1

0 em 0 pontos



O que significa HTML?

Resposta Selecionada:



c.

HyperText Markup Language

Pergunta 2

0 em 0 pontos



Qual é a diferença entre HTML e CSS e por que eles devem ser usados juntos?

Resposta
Selecionada:



a.

HTML estrutura o conteúdo da página, enquanto o CSS define o estilo visual; juntos criam páginas organizadas e esteticamente agradáveis.

Pergunta 3

0 em 0 pontos



O que significa a sigla CSS?

Resposta Seleccionada:



Cascading Style Sheets

Pergunta 4

0 em 0 pontos



Qual é a principal importância do CSS3 no desenvolvimento web?

Resposta Seleccionada:



Controlar a aparência visual e tornar o site responsivo.

Pergunta 1

0 em 0 pontos



Qual propriedade define a direção dos itens em um container Flex?

Resposta Seleccionada:



flex-direction

Pergunta 2

0 em 0 pontos



Qual propriedade permite que os itens quebrem para a linha de baixo quando o espaço não é suficiente?

Resposta Seleccionada:



flex-wrap

Pergunta 3

0 em 0 pontos



Qual unidade do CSS é mais utilizada para dividir o espaço disponível de forma proporcional em um Grid?

Resposta Seleccionada: ☒ e.
fr

Pergunta 4

0 em 0 pontos



Qual tendência moderna permite aplicar estilos com base no tamanho do **container**, e não da tela?

Resposta Seleccionada: ☒ a.
Container Queries

Pergunta 1

0 em 0 pontos



Qual é o principal objetivo de um framework no desenvolvimento web?

Resposta Seleccionada: ☒ d.
Fornecer estrutura, padrões e componentes reutilizáveis.

Pergunta 2

0 em 0 pontos



Qual é a principal função do JavaScript em uma aplicação web?

Resposta Seleccionada: ☒ c.

Adicionar comportamento e interatividade à interface.

Pergunta 3

0 em 0 pontos



Qual prática contribui diretamente para a acessibilidade em interfaces web?

Resposta Seleccionada:



e.

Navegação possível apenas com teclado.

Pergunta 4

0 em 0 pontos



Qual tendência caracteriza o desenvolvimento web moderno?

Resposta Seleccionada:



e.

Interfaces dinâmicas, acessíveis e centradas no usuário.